An AI Research Assistant for Reading Papers

Rebuilding `AI-Research-Assistant-New` around real arXiv papers, a real language model, and output a professor can check

Kunal Jangid
Roll no. B23MT1026

CSD4020: AI Specialisation Project

November 2026

Abstract

The original repository presented itself as an AI research assistant but analysed a folder of hand-written JSON records, used placeholder language models and embeddings whenever no OpenAI key was set, and reported metrics that did not measure what their names said. This report describes the replacement: a Streamlit application that searches arXiv through its API, reads each paper's full text from arXiv's HTML rendering with BeautifulSoup, indexes it with a local embedding model, and uses an LLM on Groq's free tier to write digests, answer questions with cited passages and compare papers. It is designed around a professor's reading workflow: deciding what to read, understanding it quickly, checking what the model claims, and reusing the result in teaching and writing. On a library of 12 papers (78,625 words of extracted text), 33 of the 37 supporting quotes in the generated digests (89%) were found word for word in the source paper.

Contents

1 Starting point **1**

2 What a professor needs from such a tool **2**

3 Architecture **2**

4 Getting real papers **2**
4.1 Search and metadata: the arXiv API 2
4.2 Full text: BeautifulSoup on arXiv's HTML 3

5 Retrieval **3**

6 Using a language model on a free tier **4**
6.1 Model and limits 4
6.2 Two depths of digest 4
6.3 Structured output that can be checked 4
6.4 Quote verification 4
6.5 Answers with sources 5

7 The interface **5**

8 Visualisations and their accuracy **8**

9 Measurements **10**

10 Testing **13**

11 Limitations **13**

12 Running it **14**

1 Starting point

The repository contained four entry scripts (`fixed_assistant.py`, `interactive_assistant.py`, `docker_assistant.py`, `docker_interactive.py`), a plotting module, a Docker set-up and a `research-db/` folder of paper records. Running it and reading the code showed five problems.

The AI was a placeholder. Without a key beginning `sk-`, the code fell back to LangChain's `FakeListLLM`, which returns canned strings in rotation, and `FakeEmbeddings`, which returns random vectors. The "insights" a user saw were therefore not derived from the papers.

The papers were not real. Papers were loaded only from JSON files in `research-db/`, written by hand with fields such as `keywords` and `citations`. There was no search and no way to add a real paper.

The metrics did not measure their names. "Collaboration index" was papers divided by authors, capped at 1; "trend velocity" was the number of papers divided by ten; "diversity" was the ratio of distinct keywords to keywords; "impact" averaged the hand-entered citation field.

Docker could not run. The Dockerfile was built on a Terminal-Bench base image, and the compose file ran a `solution.sh` that is not in the repository, with log paths taken from benchmark environment variables.

A credential was committed. `docker-compose.yaml` contained a literal `LANGCHAIN_API_KEY` value. It should be treated as leaked and revoked by its owner; deleting the file does not remove it from the Git history.

All of these files were removed. What replaced them is described below.

2 What a professor needs from such a tool

New papers appear faster than anyone can read them. A professor typically has to:

1. **triage**: notice the few new papers that matter to their research, out of hundreds a week;
2. **understand** a chosen paper quickly, sometimes from outside their own speciality;
3. **check** anything a summary claims before repeating it to students or in a review;
4. **reuse** what they read: in a lecture, a reading group, a referee report or a related-work section.

A language model helps with all four, but only if its output can be traced to the paper. That principle shaped the design: every generated statement is either tied to quoted text that the application checks, or to numbered passages that are shown next to the answer. Figures on the Insights page are computed, never generated.

3 Architecture

The code is a small Python package used by five Streamlit pages (Figure 1). Each module has one job, and nothing in the package imports Streamlit except `ui.py`, so every module is tested without a running app. A saved paper is one JSON file (metadata, extracted sections, digests, notes, status) and one compressed NumPy file (passage vectors). There is no database server: a reading library holds hundreds of papers, not millions, and an exact matrix product over them takes milliseconds.

Need	Feature	Where
Triage	Search ranked by similarity to a written statement of research interests; a watchlist of followed searches; a briefing that rates each new paper high/medium/low for this reader with a reason	Discover
Understand	Digest at two depths, written for a specialist, a neighbouring field or a graduate student; background needed; questions answered from the paper	Reader
Check	Each key result carries a quote that is matched against the paper text; answers cite numbered passages shown beside them; full extracted text is browsable	Reader
Reuse	Discussion questions for a reading group; side-by-side comparison; related-work paragraph with numbered references; BibTeX and Markdown export; personal notes, tags and reading status	Reader, Synthesize, Library
Overview	Timeline, categories, characteristic terms, topic map, co-author network, citation counts	Insights

Table 1: The reading workflow and the feature that serves each step.

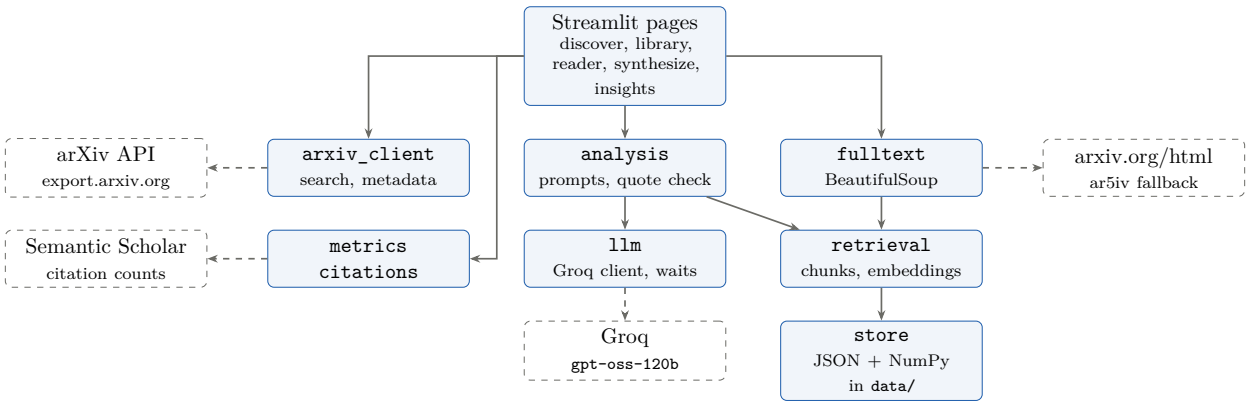


Figure 1: Modules of the `research_assistant` package and the services they call. Dashed boxes are external. Embeddings are computed locally; only the LLM call leaves the machine with paper text.

4 Getting real papers

4.1 Search and metadata: the arXiv API

`arxiv_client.py` builds queries for the Atom API at `export.arxiv.org/api/query`. Quoted phrases are kept as phrases, other words are joined with `AND`, a search can be limited to title, abstract, author or all fields, and to one of sixteen arXiv categories, and results can be sorted by relevance, submission date or last update. The feed is parsed with the standard library's XML parser into a `Paper` record: identifier and version, title, authors, abstract, dates, primary and secondary categories, comment, journal reference and DOI. arXiv reports query errors as a feed entry rather than an HTTP status; those are raised as errors instead of being shown as a paper. Calls are spaced at least three seconds apart, as arXiv's terms of use ask, and every request carries a descriptive User-Agent.

4.2 Full text: BeautifulSoup on arXiv's HTML

Abstracts are not enough to digest a paper. arXiv renders most recent submissions to HTML with LaTeXML at `arxiv.org/html/<id>`, and the ar5iv project covers many older ones. `fulltext.py` fetches the first of the two that exists and walks the document with BeautifulSoup, relying on LaTeXML's `ltx_*` classes:

- sections (`ltx_section`, appendices) become `Section(title, text)`; subsection headings are kept inline as `##` lines so retrieved passages still say where they came from;
- mathematics is replaced by its LaTeX source from the `alttext` attribute, written as `$...$`, so formulas survive as text the model can read;
- footnotes, author blocks and navigation are removed; data tables are removed (their cells are unreadable once flattened) but their captions are kept in brackets, as are figure captions; equation tables are kept;
- the bibliography is counted, not included.

A page with fewer than 1,500 characters of body text is rejected, so a stub or an error page is never taken for a paper. When neither source has the paper it is saved with its abstract only, marked as such in the interface, and offered a retry.

5 Retrieval

Each paper is split into passages of at most 1,100 characters on paragraph and sentence boundaries, with 150 characters of overlap; the abstract is always the first passage. Passages are embedded with `BAAI/bge-small-en-v1.5` through `fastembed`, an ONNX runtime that needs no GPU and no PyTorch. The model's instruction prefix is added on the query side only, as it was trained, and vectors are L2-normalised so a dot product is the cosine similarity. The 12 saved papers produce 720 passages.

The same model ranks search results against the reader's interests: the professor writes a few sentences about their research once, and each abstract is scored by its cosine similarity to that statement. The score orders results; the interface says explicitly that it is not a percentage.

6 Using a language model on a free tier

6.1 Model and limits

The LLM is `openai/gpt-oss-120b` on Groq, called through its OpenAI-compatible endpoint with the `openai` SDK and low reasoning effort. The key is read from `GROQ_API_KEY` or

`GROQ_CLOUD_API_KEY` in `.env`, or from Streamlit secrets when deployed. Groq's free tier allows 8,000 tokens per minute, counting prompt and completion together, which shapes everything else:

- a single request carries at most about 15,000 characters of paper text and asks for at most 1,800 output tokens;
- when the limit is hit, the client reads the wait Groq asks for (the `retry-after` header, or the "try again in ..." message), waits, tells the user it is waiting, and tries again; a request that is too large for the tier is reported as such rather than retried;
- streaming is used for answers, so text appears as it is written.

6.2 Two depths of digest

A whole paper does not fit in one request, so the reader chooses:

Focused (one call): the abstract, the introduction, up to two closing sections (conclusion, discussion, limitations) and the passages retrieved for three questions — the method, the main results with numbers, and limitations — until the character budget is used.

Full paper (one call per block of about 12,000 characters, then one more): the model first takes reading notes on each block, keeping every quantitative result and copying key sentences verbatim; the digest is then written from all the notes. A 12,000-word paper needs five or six calls and, on the free tier, a few minutes.

Each digest records which text it was written from, and the interface shows it.

6.3 Structured output that can be checked

Digests, comparisons and briefings are requested as JSON objects with named fields. Model output is not always well formed, so:

- the parser accepts code fences and surrounding prose and extracts the object;
- an answer wrapped in an extra object is unwrapped; an answer missing a required field is requested once more and otherwise reported as an error — an empty box is never shown as if the model had found nothing (this fixed an intermittent empty "Where they agree" panel on the Synthesize page);
- digest fields are coerced to their types (a string where a list was asked for becomes a one-item list, a null becomes empty), so exports cannot fail on an unexpected shape.

6.4 Quote verification

Each key result in a digest has two parts: a one-sentence claim and a quote of at most 30 words that the model is told to copy exactly. The application then searches the paper's text for the quote after lower-casing, collapsing whitespace and punctuation, and dropping full stops other than decimal points. A quote with an ellipsis passes when every fragment of 15 or more characters is found. The result is shown on each card as "Quote found in the paper" or "Not found word for word" (Figure 4).

This check has a precise and limited meaning. A match shows that the quoted words are in the paper, not that the claim follows from them; the reader still has to compare the two, which is why they are displayed together. A miss usually means paraphrase rather than invention (Section 9).

6.5 Answers with sources

In the Ask view, a question is embedded, the six closest passages of the paper (or eight across the whole library) are numbered and sent with the instruction to cite them as [1], [2] and to say

so when the passages do not contain the answer. The passages are shown beneath the answer with their section names, so every citation can be followed (Figure 5). The last four turns of the conversation are included so follow-up questions work.

7 The interface

The application uses Streamlit 1.67 with top navigation between five pages, a light and a dark theme defined in `.streamlit/config.toml` (Inter for the interface, Source Serif 4 for headings, JetBrains Mono for code), and Material icons. Streamlit's bundled *developing-with-streamlit* skill was followed for layout and widget choice: segmented controls for exclusive views, pills for status filters, dialogs for destructive actions, compact status indicators for long operations, `st.write_stream` for answers, and the selected paper bound to the URL so a reader view can be bookmarked or shared.

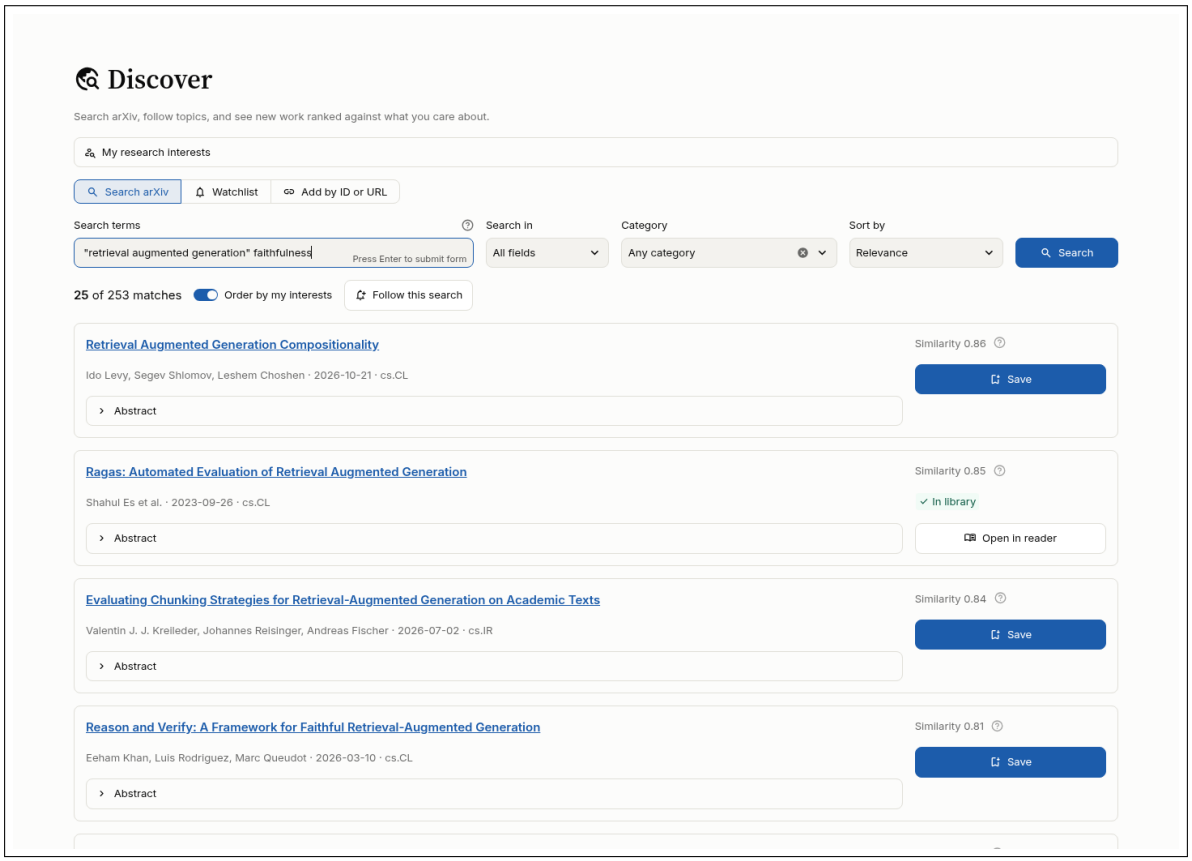


Figure 2: Discover. Results from the arXiv API, ordered by cosine similarity to the professor's interests statement. Papers already saved are marked and open in the reader.

8 Visualisations and their accuracy

The Insights page (Figures 7–9) answers "what is in my library?". Each chart was chosen for what it has to show, and each states in a caption what it measures and what it does not.

Papers over time. A bar chart of counts by month or, beyond three years, by year of first submission. Empty periods are filled with zeros so gaps are visible rather than closed up.

Categories. Horizontal bars of primary arXiv category, sorted. Each paper is counted once, so the bars sum to the number of papers.

Characteristic terms. Terms and two-word phrases ranked by mean TF-IDF weight over titles and abstracts, with English and academic stop words ("paper", "propose", "state-of-the-art", . . .)

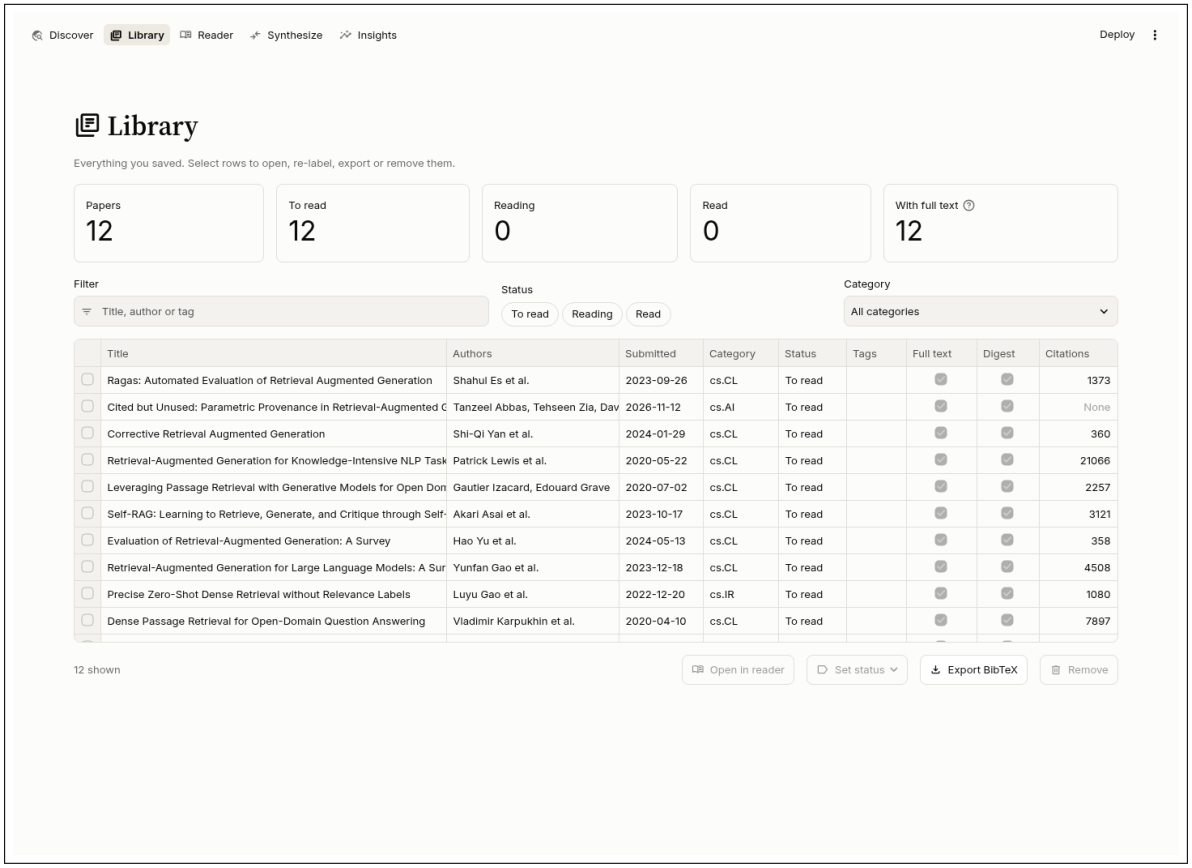


Figure 3: Library. Saved papers with reading status, tags, whether full text was extracted, and citation counts; several rows can be selected to change status, export BibTeX or remove.

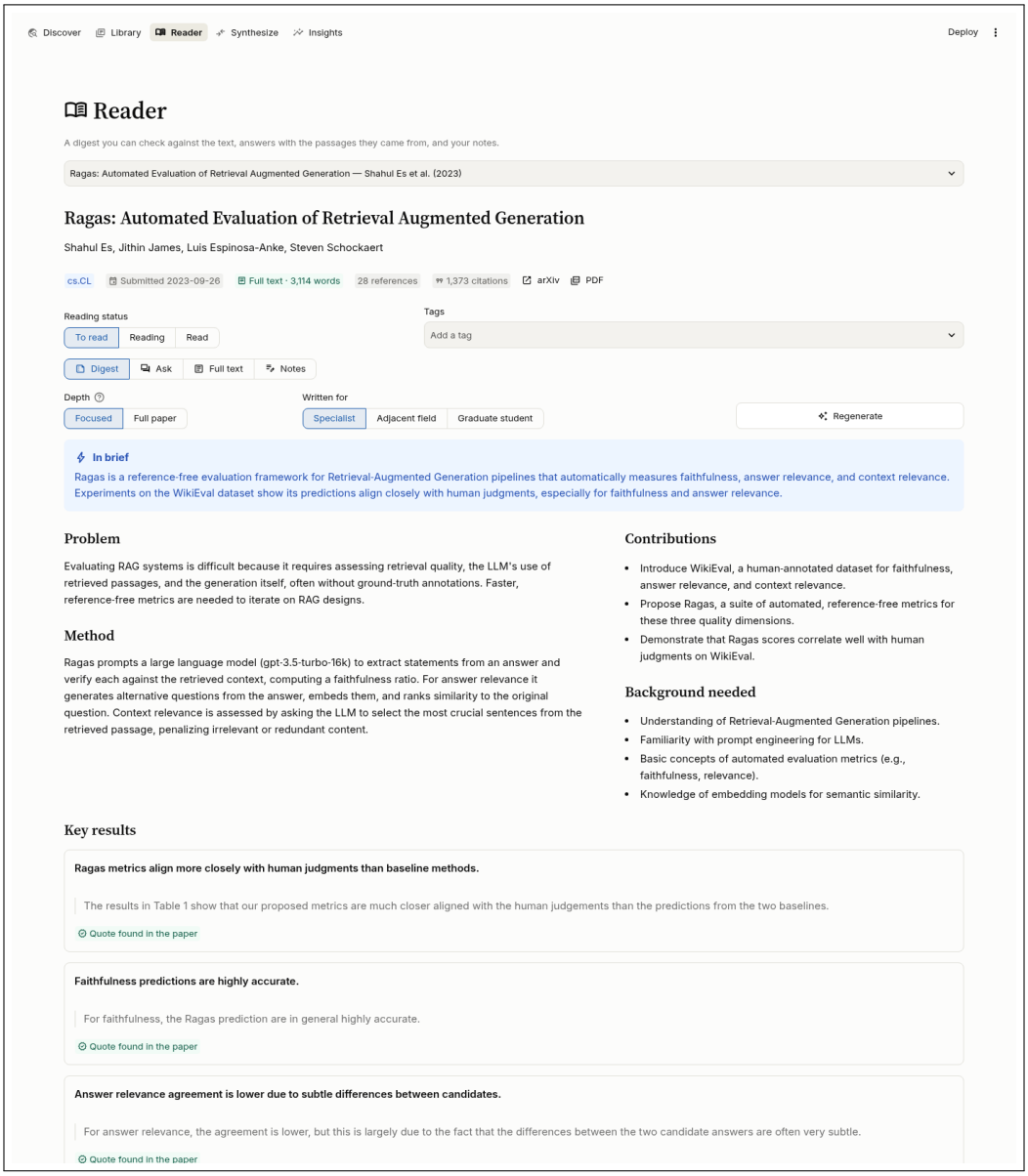


Figure 4: Reader, Digest view. Depth and audience are chosen above the digest. Each key result shows the model's claim, the quote it gave, and whether that quote was found in the paper.

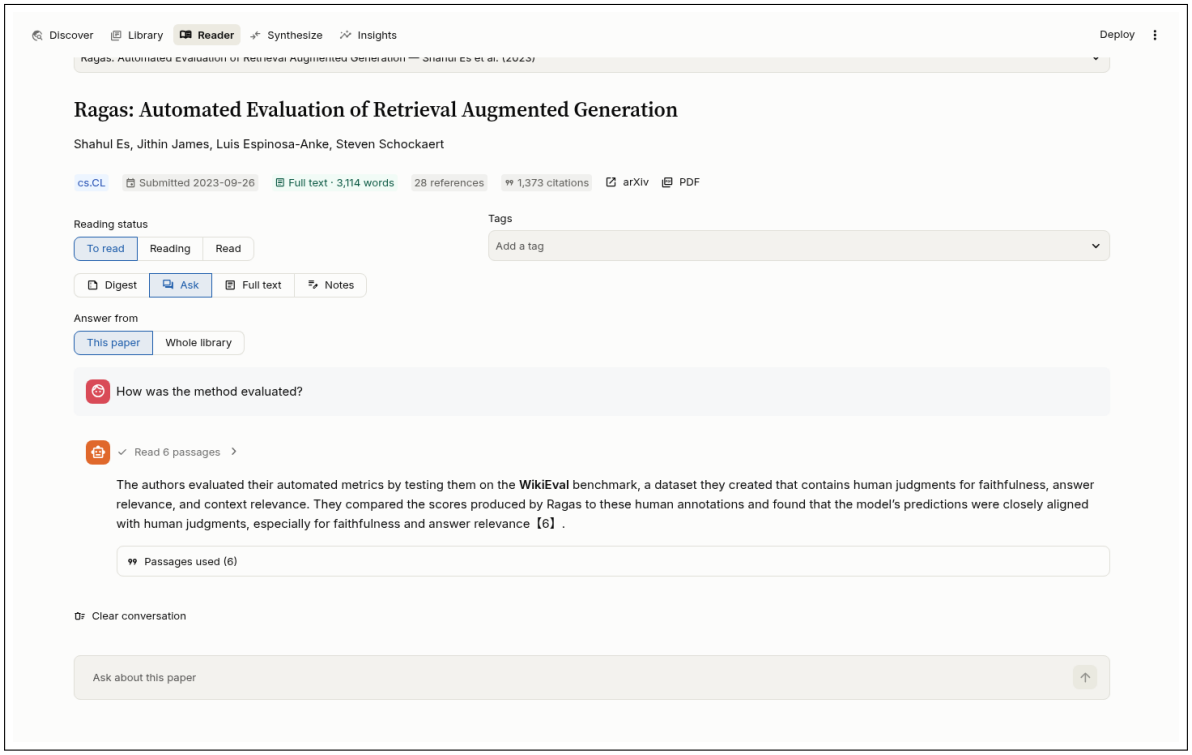


Figure 5: Reader, Ask view. The answer cites numbered passages; the passages themselves are listed below it.

removed, terms in more than 85% of papers dropped, and a term hidden when a higher-ranked phrase already contains it. Bar length is *document frequency* — the number of papers containing the term — which can be read directly, rather than a TF-IDF weight, which cannot.

Topic map. Abstract embeddings are clustered with k-means, k from 2 to 4 chosen by silhouette score, and drawn in two dimensions with PCA. Groups are labelled with their most characteristic terms and distinguished by shape as well as colour. The caption gives the silhouette score and the share of variance the projection keeps (46% for the library shown) and says that the axes have no units. With fewer than six papers the map is not drawn.

Co-author network. An interactive force-directed graph: a line joins authors of the same saved paper, thicker for more joint papers, larger dots for authors of more papers. Above it are counts that can be checked by hand: 65 authors, 212 co-author pairs, density 0.102, 11 separate groups (the largest has 17 authors), and 3 authors who appear in more than one paper.

Citations. Counts from Semantic Scholar, refreshed on demand. Citation counts are heavily skewed — from 217 to 202,845 here — so when the largest is more than 50 times the smallest the chart switches to dots on a logarithmic axis with ticks at powers of ten and exact values as labels; otherwise it uses bars. The caption notes that older papers have had longer to be cited and that this is not a quality ranking.

Colour. Categorical colours follow a fixed order and stay attached to their category; the palette was checked with a colour-vision-deficiency validator in both light and dark mode and passed (lightness band, chroma, adjacent-pair separation under simulated deuteranopia, protanopia and tritanopia, and contrast against the surface). The dark theme uses its own colours checked against the dark surface, not an inversion. Text in charts uses the theme's ink, never the series colour; a check of the dark theme found the citation labels drawn in Vega's default black, and they now take the theme's text colour.

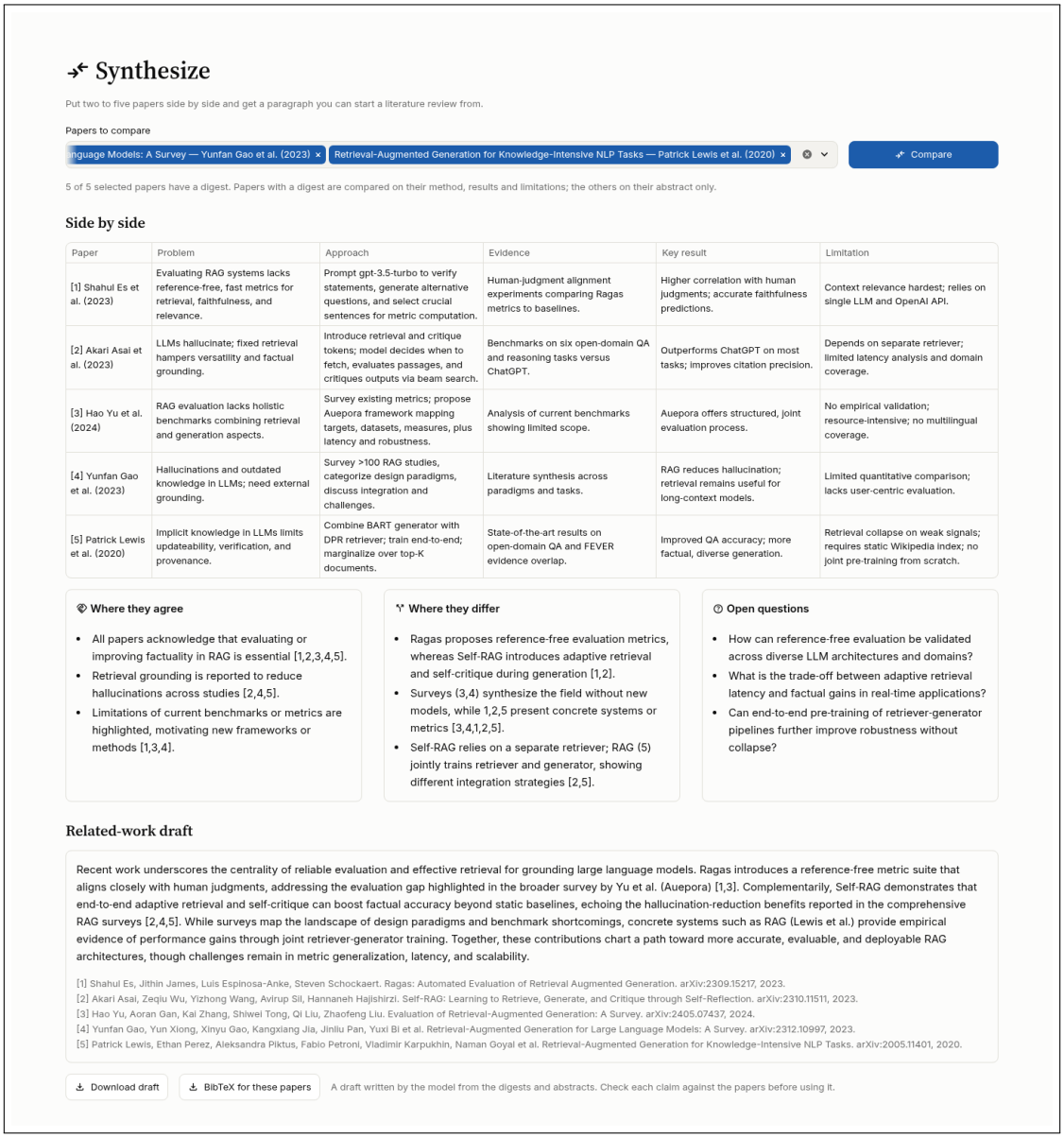


Figure 6: Synthesize, comparing five saved papers: a side-by-side table, agreements, differences and open questions, and a related-work paragraph with its numbered references. Both the paragraph and the BibTeX can be downloaded.

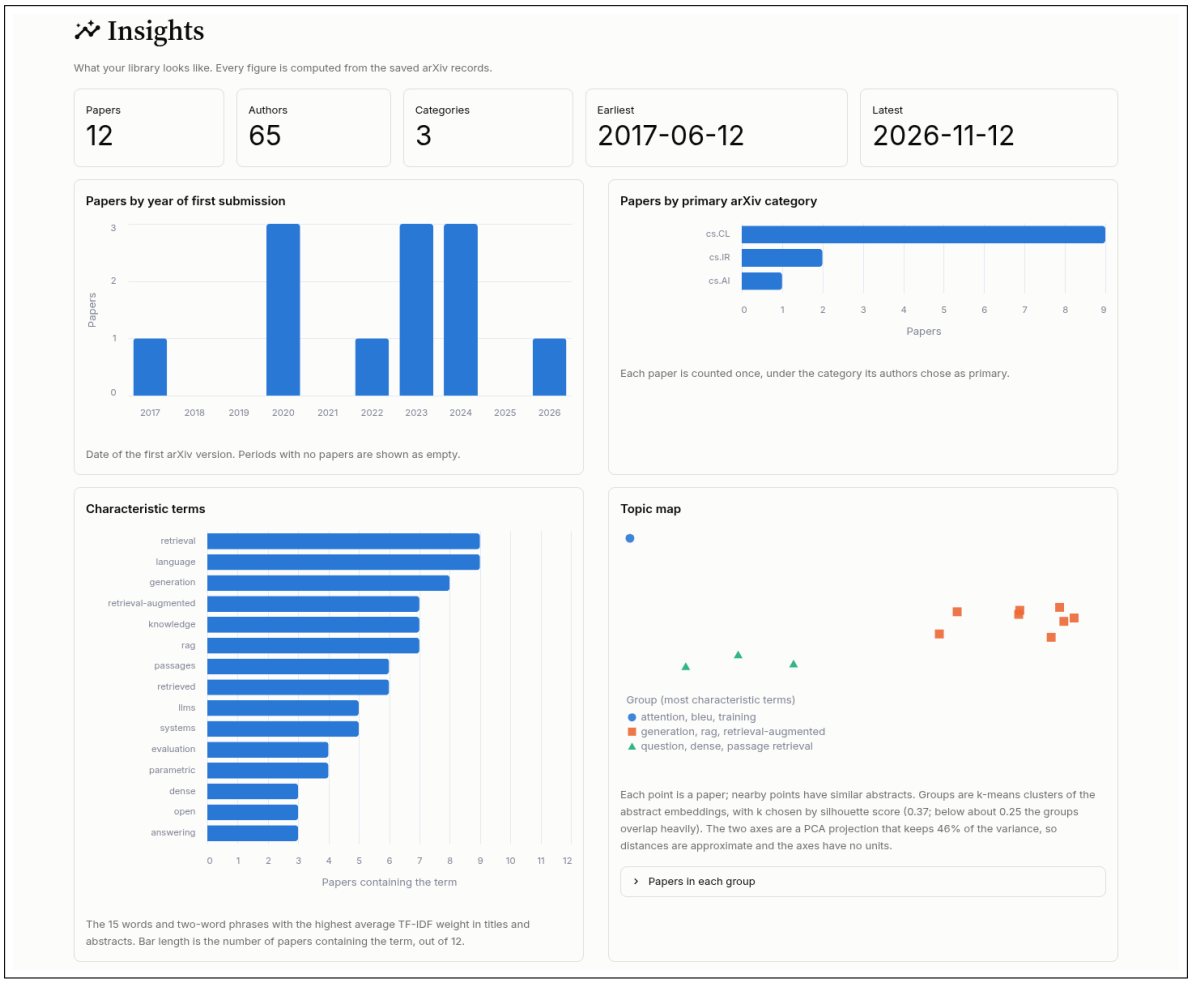


Figure 7: Insights, upper half: counts by year and category, characteristic terms by document frequency, and the topic map with its silhouette score and retained variance stated.

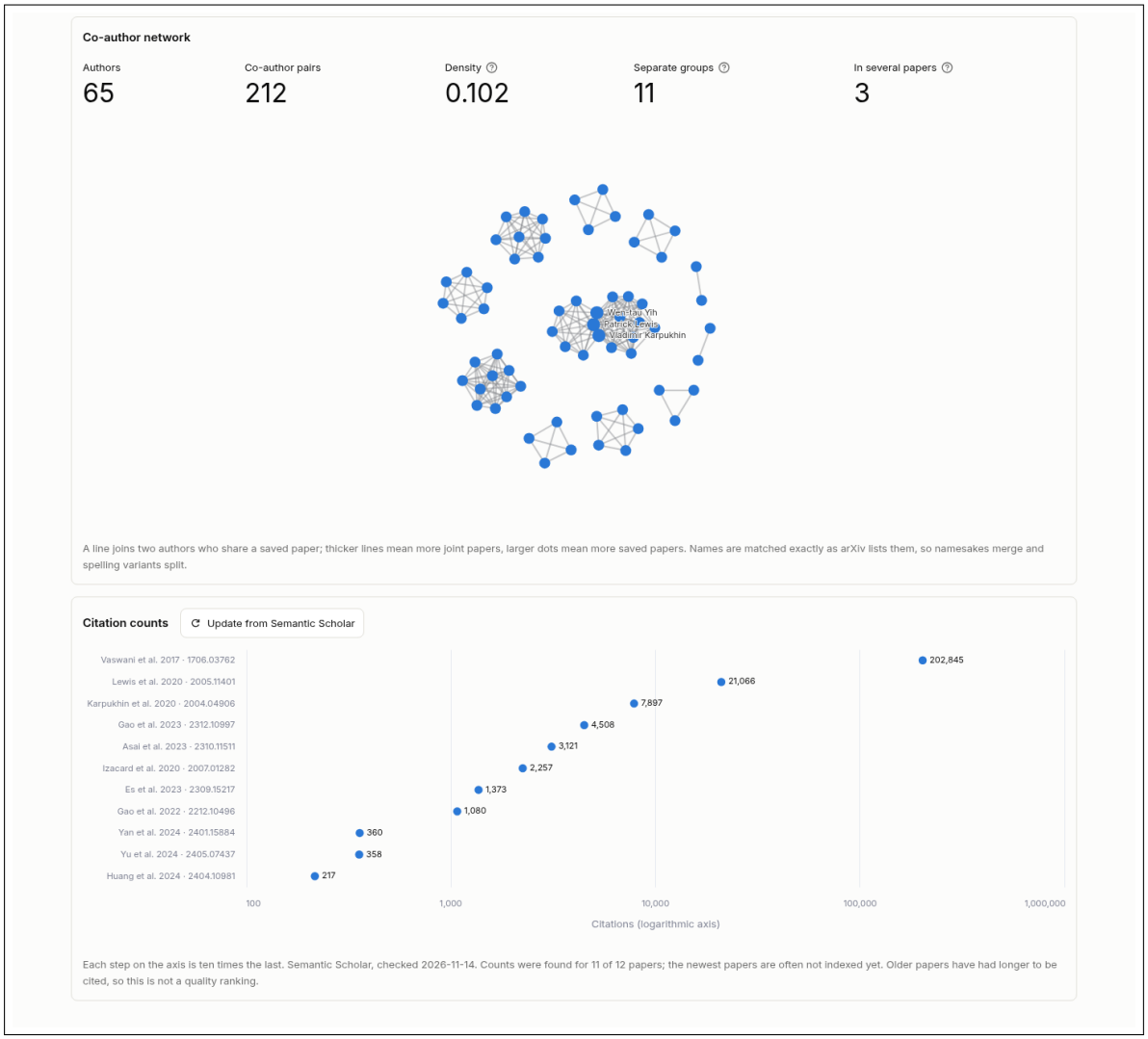


Figure 8: Insights, lower half: the co-author network with its summary counts, and citation counts on a logarithmic axis.

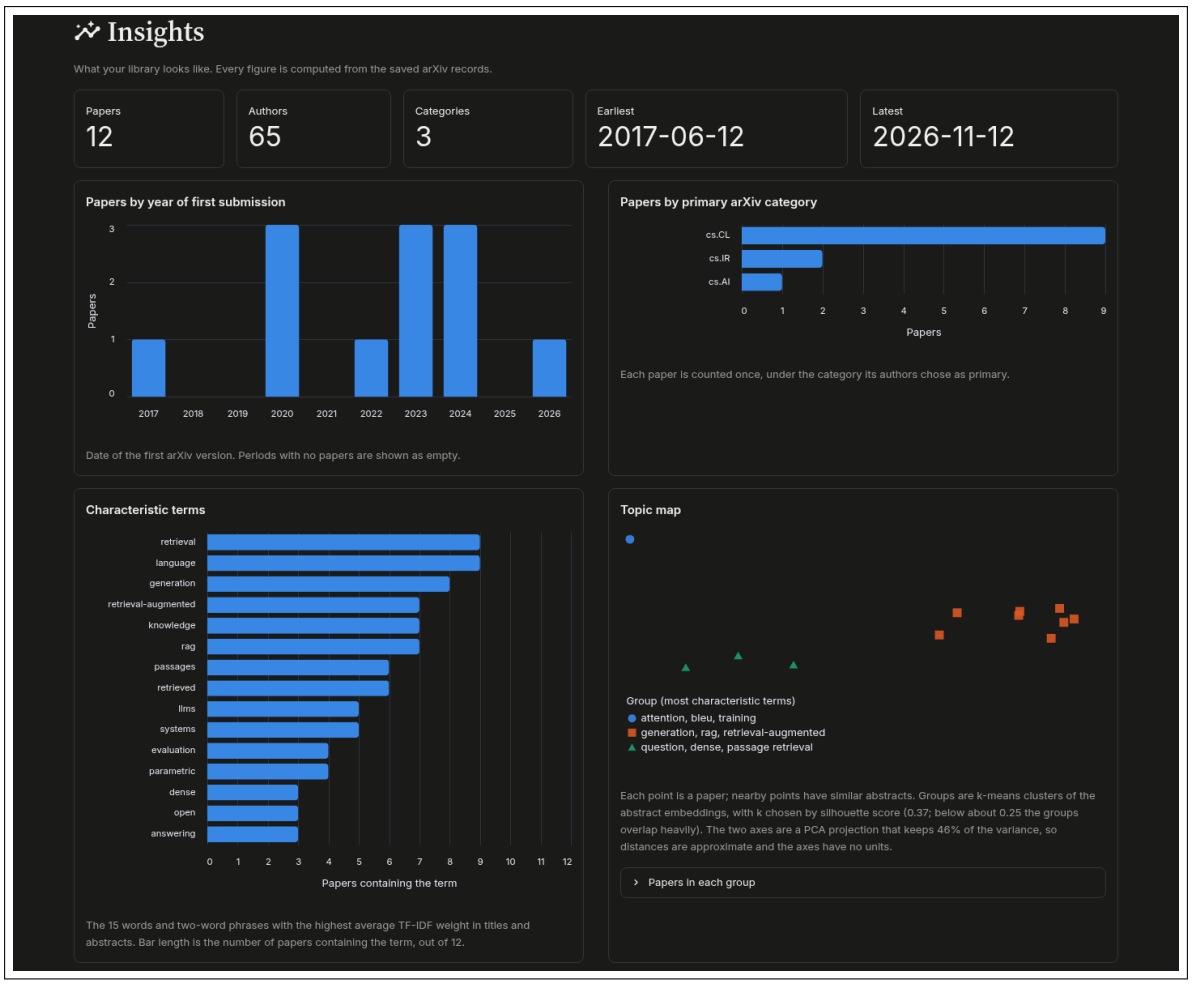


Figure 9: The same charts in the dark theme, with colours chosen and checked for the dark surface.

9 Measurements

`report/measure.py` produced the numbers in this section from the library used for the screenshots: 12 papers on retrieval-augmented generation and its foundations, from 2017 to 2026, each saved through the application. It writes `measurements.json` and the `measurements.tex` file that this report reads, so the figures here are the script's output, not copied by hand.

arXiv	Title	Year	Sections	Words	Passages	Refs	Citations	Quotes found	Digest (s)
1706.03762	Attention Is All You Need	2017	8	4,113	34	40	202,845	3/3	2.7
2004.04906	Dense Passage Retrieval for Open-Domain …	2020	13	6,293	52	48	7,897	3/4	8.9
2005.11401	Retrieval-Augmented Generation for …	2020	17	6,409	52	68	21,066	3/4	30.6
2007.01282	Leveraging Passage Retrieval with …	2020	5	2,331	19	34	2,257	3/3	24.8
2212.10496	Precise Zero-Shot Dense Retrieval without …	2022	7	3,359	26	39	1,080	4/4	35.9
2309.15217	Ragas: Automated Evaluation of Retrieval …	2023	7	3,114	25	28	1,373	3/3	–
2310.11511	Self-RAG: Learning to Retrieve, Generate, …	2023	11	12,443	187	58	3,121	3/3	31.0
2312.10997	Retrieval-Augmented Generation for Large …	2023	8	9,454	78	182	4,508	3/3	28.9
2401.15884	Corrective Retrieval Augmented Generation	2024	8	6,124	49	41	360	2/2	28.7
2404.10981	A Survey on Retrieval-Augmented Text …	2024	10	11,542	94	166	217	1/3	30.0
2405.07437	Evaluation of Retrieval-Augmented …	2024	6	5,880	46	77	358	2/2	28.1
2611.17161	Cited but Unused: Parametric Provenance in …	2026	20	7,563	58	53	–	3/3	27.6
Total			120	78,625	720			33/37	

Table 2: Per-paper results. *Words* and *Sections* are of the text extracted from arXiv HTML; *Refs* counts bibliography entries; *Citations* are from Semantic Scholar (– = not indexed yet); *Quotes found* is how many supporting quotes in the focused, specialist digest were found in the paper; *Digest* is wall-clock time including any rate-limit wait (– = generated before timing was added).

Extraction. Full text was extracted for all 12 papers: 120 sections and 78,625 words, from 2,331 words for a short paper to 12,443 for Self-RAG with its appendices. The 2026 paper was published days before the measurement, so it had no citation record yet.

Quote verification. 33 of 37 quotes (89%) were found in the paper. Each of the four misses was traced to its source. In every case the sentence exists in the paper and the claim matches it, but the model altered a word while copying: "by showing that" became "show that" (arXiv:2404.10981), an abbreviation "(CoT)" was dropped inside an ellipsis quote (same paper), and in 2005.11401 and 2004.04906 the words joining the quote to its sentence were rewritten. The check is strict on purpose: it cannot tell a harmless slip from a fabricated quote, so it flags both and leaves the judgement to the reader, who sees the quote next to the claim.

Time. 11 digests were timed, taking 2.7 to 35.9 seconds (median 28.7, mean 25.2). These times are dominated by the free tier rather than the model: the generation run hit the token limit 12 times and waited 250 seconds in total. The first digest, made with a fresh token budget, took 2.7 seconds, which is what a focused digest costs when the per-minute budget is not exhausted — for example when a reader opens papers one at a time.

10 Testing

`python -m pytest -q` runs 40 tests in about three seconds without network access or an API key.

- **Unit tests** cover arXiv identifier parsing and query building, Atom feed parsing including error entries, HTML extraction (mathematics, captions, dropped tables, nested tables, minimum length), chunking, quote matching (case, punctuation, decimals, ellipses), digest normalisation, the retry for incomplete model answers, JSON parsing from fenced or wrapped output, each metric (gap filling, category counts, terms, the topic map's minimum size, network statistics), and the BibTeX and Markdown exports.

- **Page tests** use Streamlit's `AppTest` to render every page with an empty and a populated library and to exercise filters, the reader's views, status changes and notes; the Insights test asserts that five Vega-Lite charts and one ECharts graph are drawn.

Beyond the tests, the running application was driven end to end through Streamlit's agent interface and with a headless browser: searching arXiv, saving papers, generating digests at both depths, asking questions, comparing papers, briefing on a watchlist and refreshing citations, in both themes. The screenshots in this report come from those runs.

11 Limitations

- **Full text requires an HTML rendering.** Papers that arXiv cannot render and ar5iv does not cover are kept with their abstract only. PDFs are not parsed.
- **Tables and figures are not read,** only their captions. A result reported only in a table can be missed by a digest, and a quote taken from a table is flagged as not found.
- **A found quote is not a verified claim.** The check confirms the words are in the paper; the reader has to judge whether the claim follows.
- **The comparison and the related-work paragraph are drafts.** They are written from digests and abstracts, not full texts, and the page says so.
- **Authors are matched by name,** so namesakes merge and spelling variants split in the network.
- **External services.** Semantic Scholar's anonymous API is rate-limited and lags new papers; the free Groq tier makes full-paper digests take minutes.
- **Single user.** The library is a folder of files on one machine. Deploying for a group would need per-user storage.

12 Running it

```
uv venv --python 3.12 && uv pip install -r requirements.txt
cp env.example .env # set GROQ_API_KEY (or GROQ_CLOUD_API_KEY)
source .venv/bin/activate
streamlit run streamlit_app.py
python -m pytest -q # 40 tests, no network needed
python report/measure.py # regenerate the numbers in this report
```

Dependencies are pinned in `requirements.txt`. The embedding model (about 65 MB) is downloaded once into `data/models` the first time a paper is saved.